Intrebari SO 2022-2023

Diana-Iuliana CISMARU

March 27, 2023

1 Informatii pe care le consider utile

Sistemul de operare este **componenta software** folosita pentru a asigura buna functionare a sistemului. Are rolul de arbitru si gardian al resurselor sistemului. Aplicatiile trebuie sa aiba acces sigur (izolat), echitabil (sa nu acapareze o singura aplicatie) la resursele sistemului.

Daca o aplicatie are permisiune sa faca un syscall, sistemul va permite acest lucru. In caz contrar, SO va trimite o eroare de tip **Nop** (NO permission).

Codul serial se refera la codul care ruleaza pe un singur procesor sau nucleu, in ordine secventiala, fara a se paraleliza. Acest tip de cod nu poate folosi avantajele paralelizarii pentru a imbunatati performanta sistemului.

Mutexurile sunt mecanisme de sincronizare utilizate pentru a controla accesul la resursele comune. Scopul lor principal este de a preveni race condition-urile, care apar atunci cand mai multi procese sau thread-uri incearca sa acceseze aceeasi resursa in acelasi timp. Un mutex poate fi "incuiat" de un proces sau thread, astfel incat alte procese sau thread-uri sa nu poata accesa acea resursa pana cand mutex-ul nu este "deblocat".

Semafoarele sunt, de asemenea, mecanisme de sincronizare utilizate pentru a controla accesul la resursele comune. in loc de a permite accesul la o resursa in exclusivitate, ca mutexurile, semafoarele permit multiple procese sau thread-uri sa acceseze o resursa intr-un mod controlat. Un semafor poate fi incrementat sau decrementat, permitand sau restrictionand accesul la resursa. Semafoarele pot fi utilizate pentru a implementa diverse strategii de sincronizare, cum ar fi gestionarea buffer-ului sau producer-consumer.

Un **socket** este un punct final de comunicare intre doua aplicatii pe aceeasi sau pe diferite dispozitive, utilizat pentru a transmite si primi date in retea. Este un mecanism de comunicare care permite aplicatiilor sa se conecteze la o retea si sa transmita informatii intre ele, folosind protocolul de comunicatie specific, cum ar fi TCP sau UDP.

2 Software Stack

2.1 Ce este un apel de sistem?

Interfata expusa de SO pentru actiuni privilegiate se numeste interfata de sistem. Un element din aceasta interfata este apelul de sistem, presupunand **trecerea din user space in kernel space** (domain switch). Asigura o metoda programatica prin care **se cere un serviciu de la nucleu**. Apelurile de sistem sunt, in general, referite printr-un nume si sunt asociate cu un numar. Se spune ca un apel de sistem este un **trap** in modul privilegiat.

2.2 De ce sunt necesare apeluri de sistem?

- pentru a extinde functionalitatile kernelului
- pentru a executa operatii privilegiate (lucrul cu memoria, comunicarea cu dispozitivele de I/O)
- pentru izolarea kernelului de restul aplicatiilor, prin partitionarea pe ring-uri

Nota: Partitionarea pe ring-uri este o tehnica de securitate utilizata in sistemele de operare, care imparte drepturile de acces si permisiunile pentru resursele sistemului in mai multe niveluri. Fiecare inel are un grad diferit de acces la resursele sistemului: inelul 0 este cel cu cel mai mare nivel de acces (de ex, kernelul), iar inelul 3 este cel cu cel mai mic nivel de acces (aplicatiile utilizatorului).

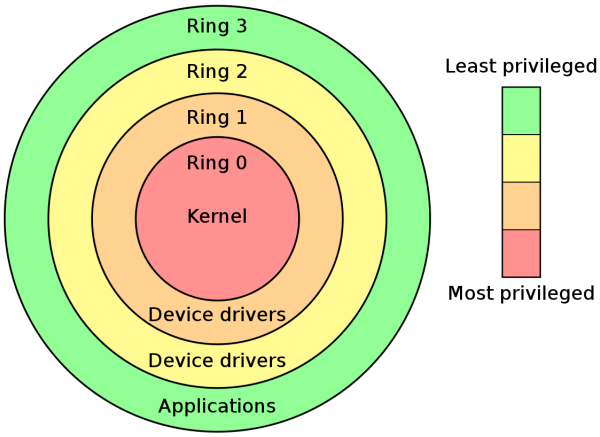


Figure 1: Protection rings

2.3 Ce avantaje/dezavantaje au apelurile de sistem?

- **Avantaj**: mentin kernelul **izolat** de restul aplicatiilor, ruland in modul privilegiat, permitand in acelasi timp **accesul optim la resursele hardware** (pentru care exista cerere mare din partea aplicatiilor);
- **Dezavantaj**: overhead-ul temporal.

2.4 Ce inseamna user/application mode/space (mod neprivilegiat)? Ce inseamna kernel/supervisor mode/space (mod privilegiat)?

- **User space**: este nivelul de privilegiu 3, in care aplicatiile utilizatorului ruleaza. Ele au **acces limitat** la resursele sistemului (precum hardware-ul sau memoria);
- **Kernel space**: este nivelul de privilegiu 0, in care ruleaza nucleul, asigurandu-se de impartirea corecta a resurselor. Acest spatiu are **acces complet** la resursele sistemului;
- separarea acestor zone ofera o protectie suplimentara sistemului, prevenind accesul neautorizat sau coruperea resurselor critice.

2.5 Cum se realizeaza tranzitia din mod neprivilegiat in mod privilegiat?

Tranzitia din user space in kernel space se realizeaza prin **apeluri de sistem,** invocate, in general, de aplicatii, prin wrappere de biblioteca.

La bootare, hardware-ul se afla in kernel mode, iar sistemul de operare se incarca si aplicatiile pornesc in user mode. Cand apare o intrerupere (*trap*), se trece in mod privilegiat.

In momentul in care se invoca un apel de sistem, se trece din user mode in kernel mode si bitul de mod devine 0. Se executa rutina asociata syscall-ului si apoi se reintra in user mode, bitul de mod devenind din nou 1.

2.6 Ce se intampla in momentul tranzitiei in mod privilegiat? Cum se/Cine asigura (enforcement) existenta modului privilegiat?

rapspuns asemanator cu cel anterior

O aplicatie face o cerere, sistemul de operare verifica daca aplicatia are permisiunile necesare, apoi realizeaza actiunea ceruta si intoarce rezultatul.

Distinctia dintre kernel space si user space este asigurata de modurile de privilegiu ale procesorului, marcate printr-un bit. Acesta este salvat in registrul CPL (Current Privilege Level) pe x86 sau CPSR pe ARM.

2.7 Ce este o biblioteca?

Este o colectie de functii precompilate. Incorporeaza componente software reutilizabile. Atunci cand un program are nevoie de o functie, linker-ul va apela respectiva functie din biblioteca.

2.8 Care este diferenta dintre o aplicatie si o biblioteca?

Biblioteca nu poate fi executata singura. O aplicatie, da.

2.9 Care este asocierea apel de biblioteca / apel de sistem?

Un apel de biblioteca presupune zero (*strcpy*), unul (*write*) sau mai multe apeluri de sistem (*popen*). Exista apeluri de sistem care sunt invocate de mai multe apeluri de biblioteca (ex: fopen si open apeleaza syscall-ul open).

De exemplu, pentru un *scanf*, se va face un *read* in spate doar daca datele ce trebuie citite nu sunt deja in bufferul mentinut de libc de la un read anterior.

2.10 Ce este entry point-ul intr-un executabil?

Este adresa primei instructiuni. In general, refera un simbol cu numele *_start*. Cand loader-ul il va incarca in memorie, va seta instruction pointer-ul la valoarea din entry-point.

2.11 Care este rolul bibliotecii standard C (libc)?

Ofera o suita de functii, atat pentru folosirea integrala in user space (precum *strlen()*, cat si pentru interactiunea cu kernelul (wrappere peste apelul de sistem, precum *printf()* sau *scanf()*).

2.12 Ce actiuni se pot executa doar in mod privilegiat?

- alocarea memoriei
- operatii cu cache-ul
- interactiunea cu I/O-ul
- intreruperi
- comunicarea cu alte procese
- configurarea de permisiuni

2.13 Ce operatii / instructiuni low-level (ISA) se pot executa doar in mod privilegiat?

- instructiunea **INT**: cheama o intrerupere software
- **HLT**: opreste procesorul pana la aparitia unei intreruperi sau a unui semnal

2.14 Ce este un sistem de operare monolitic?

Este un SO care are user services si kernel services implementate in acelasi spatiu de adrese. Deoarece totul este intr-un singur loc, suprafata de atac este mai mare. Exemplu: Unix

2.15 Ce este un sistem de operare de tip microkernel?

Este un SO care are user services si kernel services implementare in locuri diferite in memorie. Este un SO mic, kernelul fiind mic.

2.16 Care sunt avantajele unui sistem de operare monolitic?

- **Avantaje**: viteza de executie mare, overhead mic;
- **Dezavantaje**: securitatea potential mai slaba: intrucat kernelul este mai mare, si suprafata de atac este mai mare (spunem ca are TCB - Trusting Computing Base - mai mare). Este mai putin flexibil, deoarece atunci cand se face o modificare asupra unei componente, trebuie schimbat tot.

2.17 Care sunt avantajele unui sistem de operare de tip microkernel?

- **Avantaje**: se extinde mai usor. Daca un serviciu pica, nu afecteaza si alte servicii. Este mai sigur (TCB redus), datorita modularitatii;
- **Dezavantaje**: este mai lent, din cauza comunicarii intre servicii.

2.18 Care tip de sistem de operare are mai multe apeluri de sistem?

Cel de tip microkernel, deoarece sunt mai multe tranzitii user-kernel pentru a permite componentelor software care ruleaza in user mode sa interactioneze.

2.19 Care este avantajul folosirii masinilor virtuale din perspectiva securitatii?

Masinile virtuale (inclusiv SO-ul acestora) nu fac parte din TCB. Sunt un mediu relativ izolat.

2.20 Ce este o biblioteca statica? Ce este o biblioteca dinamica?

- **Biblioteca statica:** (.a pe Linux, .lib pe Windows) este o colectie de functii ce sunt copiate direct in executabilul programului, la link-time.
- **Biblioteca dinamica:** (.so pe Linux, .dll pe Windows) este o colectie de functii ce sunt aduse in executabil la load-time, printr-un segment partajat.

2.21 Cand preferam folosirea static linking, respectiv dynamic linking? Cu ce difera un executabil dinamic de un executabil static?

- **Executabilele statice:** sunt portabile, pot fi transferate pe un alt sistem fara a fi nevoie de prezenta unor biblioteci. Lansarea lor in executie este mai rapida, nefiind nevoie de rezolvarea referintelor (**dynamic binding**): toate referintele au fost rezolvate la link-time. Totusi, executabilul static e mai mare ca dimensiune, deoarece are toate bibliotecile incluse.

  Exemplu: un program care doar printeaza "Hello world!" are aproximativ 800K, pentru ca el trebuie sa aduca tot printf-ul si toate functiile folosite de printf;

- **Executabilele dinamice:** ocupa mai putin spatiu pe disc, iar procesele aferente ocupa mai putina memorie fizica, partajand zonele read-only (.text, .rodata) din biblioteci cu alte procese obtinute din executabile dinamice care folosesc aceleasi biblioteci.

2.22 Dati exemplu de apel de sistem blocant.

- **read()**: se blocheaza pana poate fi citit cel putin un byte
- **write()**: se blocheaza pana s-a putut scrie intregul buffer

2.23 Dati exemplu de apel de sistem neblocant.

- **open()**, **fork()**
- **wait()**: este blocant pana cand procesul copil isi incheie executia, dar neblocant cand procesul copil este deja mort in timpul apelarii

2.24 De ce, in general, o aplicatie trebuie sa execute un apel de sistem pentru a accesa un dispozitiv hardware? De ce NU poate accesa direct dispozitivul hardware?

Pentru ca, in sistemele de operare care nu sunt unikernele, ruleaza in acelasi timp mai multe aplicatii. Majoritatea acestora doresc sa interactioneze cu hardware-ul, aparand astfel probleme cu accesul concurent la resurse.

Sistemul de operare, prin syscalls (poate fi privit drept o cerere la un ghiseu), mediaza accesul, fortand aplicatiile sa nu se calce in picioare.

2.25 Ce intelegem prin overhead spatial si overhead temporal?

- **spatial**: dimensiunea suplimentara sau spatiu aditional necesar pentru realizarea unei anumite operatii;

  de exemplu, un algoritm poate avea un overhead spatial mare din cauza alocarii unui spatiu de stocare suplimentar pentru o structura de date complexa;

- **temporal**: timpul suplimentar necesar pentru realizarea unei operatii;

  de exemplu, un algoritm poate avea un overhead temporal mare din cauza unui numar mare de operatii

2.26 Dati exemplu de mecanism / functie care reduce overhead-ul spatial si unul care reduce overhead-ul temporal.

Mecanismul de zero-copy reduce overhead-ul temporal, prin evitarea domain switch-urilor. In plus, reduce si overhead-ul spatial, deoarece nu se mai face double buffering.

Mecanismul de shared memory reduce overhead-ul spatial, prin accesarea directa a bufferului din kernel space.

2.27 Ce inseamna double buffering? in ce situatie concreta (mecanism/-functie) apare?

Este o tehnica prin care doua buffere sunt folosite pentru cresterea performantei. Astfel, procesarea unor date se face independent de aducerea altora: in timp ce se lucreaza cu datele dintr-un buffer, se pot pregati date in altul.

Exemplu: SO vrea sa scrie date catre disk. Dar disk-ul este lent, asa ca, pentru a nu mai astepta dupa el, scrie intr-un buffer.. Problema: bufferul este plin si nu mai poate sa il foloseasca pentru altceva. Solutia? Double buffering: mai ia inca un buffer ca sa aiba unde sa scrie pana cand discul ia ce ii trebuie.

2.28 Ce se intampla cand exista o eroare critica (de tip Segmentation fault) la nivelul sistemului de operare?

Daca in nucleu se ajunge la Segmentation Fault, atunci SO intra in kernel panic.

2.29 Dati exemplu de un apel de biblioteca standard C care nu cauzeaza apel de sistem.

strlen(), strcpy()

2.30 Dati exemplu de un apel de biblioteca standard C care cauzeaza apel de sistem.

scanf(), open()

2.31 Care este avantajul si dezavantajul existentei unei stive software la nivelul unui sistem de calcul?

- **Avantaje**: izolare buna a aplicatiilor, portabilitate, modularitate;
- **Dezavantaje**: overhead temporal.

2.32 Care este avantajul si dezavantajul folosirii unui limbaj de programare high-level (precum Java, Python, Lua)?

- **Avantaje**: management automat al memoriei, mai usor de utilizat, portabilitate mare;
- **Dezavantaje**: overhead temporal, control limitat asupra hardware-ului.

2.33 Care este avantajul si dezavantajul folosirii unui limbaj de programare low-level (precum C, C++, Rust, D)?

- **Avantaje**: control mare asupra hardware-ului, rapiditate;
- **Dezavantaje**: prone to mistakes - mai dificil de inteles si de utilizat, dezvoltare mai lenta si mai putin productiva, portabilitate redusa.

3 Data

3.1 Cum asigura sistemul de operare separatia intre procese?

Prin mecanismul de **memorie virtuala**. Fiecare proces are impresia controlului intregii memorii. Astfel, ii este imposibil sa acceseze memoria altui proces.

3.2 Ce inseamna mecanismul de memorie virtuala?

Este un mecanism utilizat de nucleul SO pentru a **gestiona eficient memoria**, prin izolarea intre procese si prin impartirea memoriei fizice intre acestea. Fiecare proces lucreaza cu memorie virtuala, motiv pentru care acesta are impresia ca are la dispozitie toata memoria.

Dezavantaj al memoriei virtuale: **overhead de calcul a adresei fizice** din adresa virtuala la fiecare acces.

Pentru a ajunge, totusi, la memorie fizica, SO **mapeaza** fiecarei pagini virtuale a unui proces (page) o pagina din memoria fizica (frame). In momentul in care un proces refera o zona de memorie, el o refera din memoria virtuala, dintr-un page. Page-ul apoi este **translatat de MMU** (Memory Management Unit) in frame-ul corespunzator si procesul obtine datele dorite.

3.3 Ce reprezinta spatiul virtual de adrese al unui proces?

Este o inlantuire de zone (segmente), folosita pentru a retine ce memorie doreste procesul sa rezerve sau sa elibereze. Spatiul virtual de adrese este **unic fiecarui proces**. El ocupa $2^{32} octeti$ (4 GB) pe un sistem de 32 de biti. Cuprinde zonele text(cod), rodata, data (date initializate), bss (date neinitializate), heap si stiva.

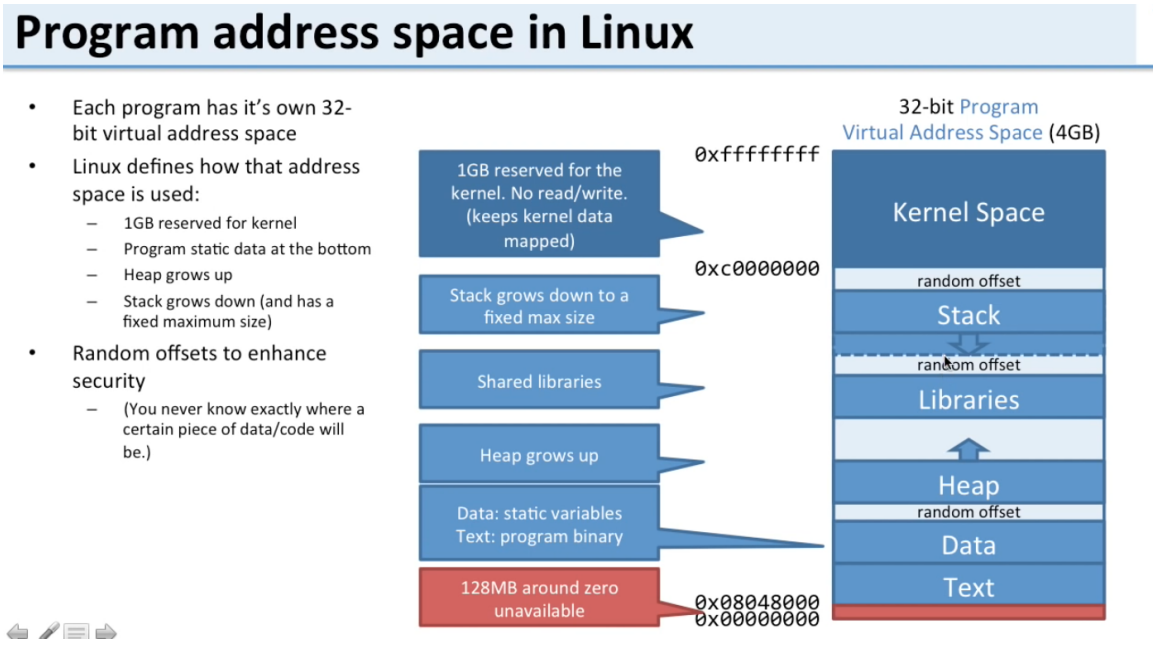


Figure 2: Virtual Address Space - VAS

3.4 Cu ce difera zona de date de zona de cod in spatiul virtual de adrese al unui proces?

Difera prin permisiunile lor, dar si prin functionalitate. Zona de date e **RW-** si retine, in general, variabilele utilizate de program. Zona de cod este **R-X** si retine instructiunile care se executa.

3.5 Ce permisiuni are zona .text/.data/.rodata/.bss/de stiva/de heap?

- .text (cod): R-X
- .data (date initializate): RW-
- .rodata (constante): R–
- .bss (date neinitializate): RW-
- stiva: RW-
- heap: RW-

3.6 Ce este paginarea memoriei?

Este mecanismul prin care **RAM-ul este impartit eficient** in mai multe bucati, numite pagini, de obicei de dimensiune 4kB. Pe un sistem de 32 de biti, un proces are 2^{20} pagini virtuale. Forma veche de translatare era segmentarea.

Avantaje: nu mai avem fragmentare externa (cauzate de segmentare);

Dezavantaje: poate aparea fragmentarea interna.

Nota: Fragmentarea interna reprezinta niste "gauri" intr-un spatiu deja alocat. Tinand cont ca la paginare se aloca bucati de dimensiune fixa, orice alocare de memorie care nu este multiplu de acea dimensiune va crea fragmentare interna. De exemplu, daca vreau sa aloc 5kB, am nevoie de 2 pagini, deci se vor aloca 8kB. Gaura de 3kB este fragmentarea interna.

3.7 Ce rol are tabela de pagini?

Are rolul de a face asocierea (maparea) dintre pagini virtuale si pagini fizice - este **indexata** cu adresa paginii virtuale si **contine** adresa paginii fizice. Contine 2^{20} intrari pe un sistem de 32 de biti.

O adresa virtuala este cuprinsa din: adresa de pagina virtuala (20 de biti) si un offset in pagina (12 biti). Fiecare intrare in tabela de pagini se numeste PTE (Page Table Entry). Fiecare proces are o unica tabela de pagini.

3.8 Ce este si ce rol are MMU (Memory Management Unit)?

MMU sau MTU este o componenta hardware de pe procesor, ce are responsabilitatea **translatarii paginilor virtuale in pagini fizice**. El poate genera page fault-uri.

3.9 Ce rol are TLB?

TLB = Translation Lookaside Buffer, este un **cache** pentru intrarile din tabela de pagini a unui proces. Pentru a micsora timpul de acces, acesta retine cele mai recent accesate intrari in tabela de pagini, si, pentru fiecare adresa virtuala, **verifica daca are adresa fizica memorata**. Daca da, atunci are loc un TLB hit, altfel rezulta un TLB miss.

3.10 Care este ordinul de marime al numarului de intrari ale TLB?

TLB-ul are intre 128 si 256 de intrari.

3.11 Ce contine o intrare in tabela de pagini?

- PFN (page frame number) / adresa fizica;
- un bit ce precizeaza daca o pagina este valida; daca nu, rezulta in exceptie de acces la memorie (page fault);
- precizari legate de tipul de operatii permise (read-only, read-write);
- un dirty bit.

3.12 Ce inseamna tabela de pagini multi-nivel (ierarhica)? De ce este utila?

Este o tabela de pagini care retine adrese ale altor tabele de pagini. A aparut din nevoia de a micsora dimensiunea tabelei de pagini. O adresa din tabela ierarhica este formata din 10 biti (page directory index) + 10 biti (PTE index) + 12 biti (offset).

O intrare in page directory refera o structura page table cu 2^{10} intrari.

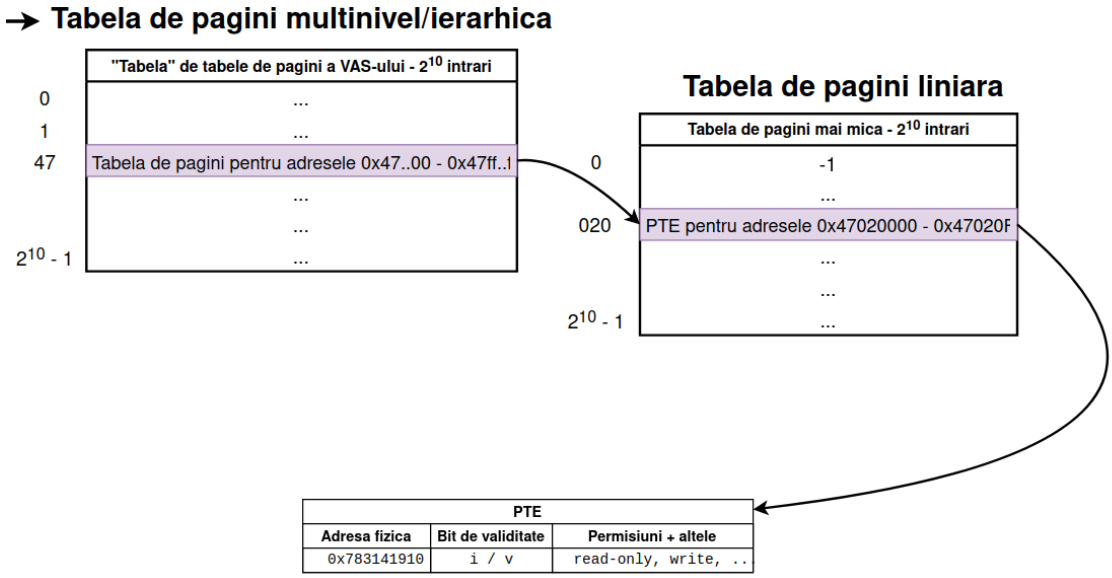


Figure 3: Tabela de pagini ierarhica

3.13 Cand are loc un TLB miss?

Atunci cand adresa paginii pe care vreau sa o accesez...

- nu a mai fost folosita;
- a fost folosita cu mult timp in urma si a fost eliminata la update-ul TLB-ului;
- nu exista in TLB - adica nu exista o adresa fizica alocata pentru cea virtuala.

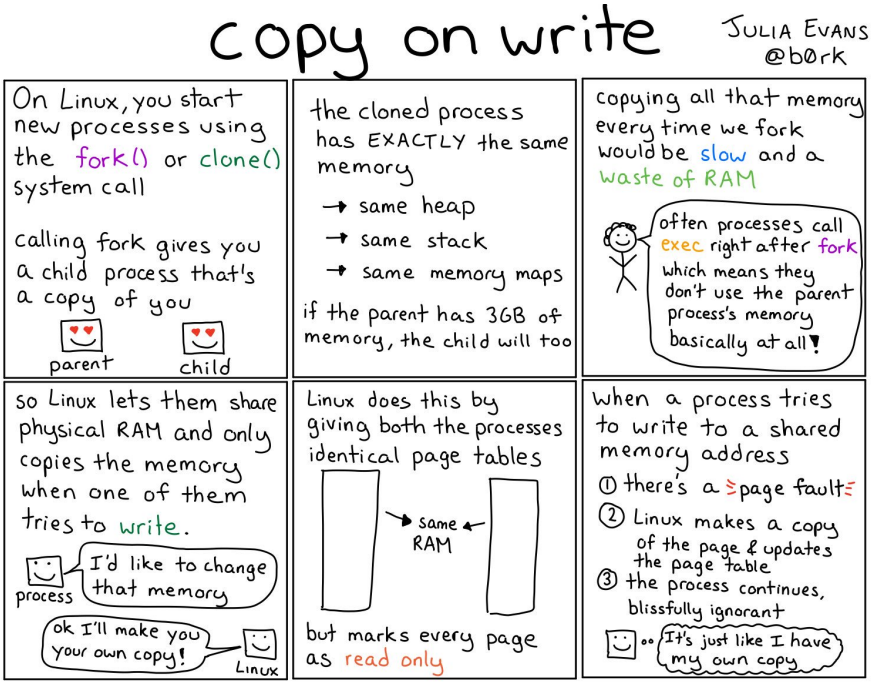
3.14 De ce se goleste TLB-ul (TLB flush) la schimbare de context?

TLB flush are loc la context switch, deoarece TLB-ul trebuie sa retina **doar informatia procesului curent**. Nu este permis accesul proceselor la informatie ce nu ar trebui sa fie partajata. Dupa context switch, TLB este repopulat pe masura ce sunt accesate noi pagini.

3.15 De ce nu este nevoie de TLB flush la schimbarea de context intre doua thread-uri ale aceluiasi proces?

Pentru ca thread-urile aceluiasi proces impart aceeasi tabela de pagini, deci folosesc si au voie sa acceseze aceleasi adrese fizice.

3.16 Ce inseamna mecanismul de copy-on-write?



Mecanismul de copy-on-write este modalitatea prin care procesul copil "imparte" corespondenta pagini-frames cu procesul parinte, pana la prima actiune de scriere, pentru a evita copierea inutila a datelor. Este o forma de operatie "lazy", similar demand paging.

Imediat dupa fork(), fiecare spatiu virtual (atat al procesului parinte, cat si al copilului) **refera acelasi spatiu fizic**. Paginile sunt marcare **read-only** in tabela de pagini. La o actiune de **scriere** a unuia dintre procese, MMU genereaza **page fault**. In informatiile interne ale SO, pagina pe care se incearca scrierea este marcata "copy-on-write". Sistemul aloca o pagina fizica noua, duplica in ea continutul paginini initiale si actuealizeaza intrarea in tabela de pagini a procesului care a realizat scrierea: pagina este marcata read-write. Acum, **scrierea se va face pe frame-ul duplicat.**

3.17 Dati exemple de situatii in care are loc mecanismul de copy-on-write.

- **virtualizare**: COW este utilizat pentru a crea snapshot-uri ale sistemului virtual, astfel incat sa poata fi restaurat la o versiune anterioara in cazul unei erori. In acest caz, datele nu sunt copiate in mod real, ci sunt marcare pentru a fi copiate doar daca sunt modificate;
- **sisteme de versionare ale fisierelor**: COW poate fi utilizat pentru a pastra versiuni anterioare ale unui fisier, fara a fi necesara copierea acestuia in mod real la fiecare modificare. In acest caz, doar diferentele intre versiuni sunt stocate.

3.18 Cu ce apel de sistem asociem copy-on-write?

Copy-on-write se asociaza cu apelul 'fork()'.

3.19 Cand se duplica o pagina marcata copy-on-write?

O pagina marcata copy-on-write se duplica cand un proces incearca sa scrie in ea.

3.20 Cine detecteaza un acces de scriere intr-o pagina marcata copy-on-write?

Handler-ul pentru page-fault al SO-ului detecteaza un acces de scriere intr-o pagina marcata COW. El duplica apoi pagina fizica in cauza, actualizeaza permisiunile si referintele.

3.21 Ce inseamna demand paging?

"Demand paging" sau "alocarea la cerere" reprezinta un mecanism prin care o pagina se poate **"rezerva"**. Atunci cand alocam o pagina virtuala noua (folosind malloc, mmap etc.) nu e obligatoriu sa alocam si un frame. Astfel, amanam alocarea efectiva a paginii fizice pana la primul acces: atunci se realizeaza si maparea.

Intern, atunci cand se rezerva prima oara un page, se marcheaza pagina ca **nevalida in tabela de pagini** si ca **valida-nemapata** in informatiile sistemului de operare. La primul acces, MMU genereaza exceptie, deoarece intrarea in tabela de pagini este nevalida. Exceptia e capturata de SO, acesta alocand o pagina fizica pe care o completeaza apoi in tabela de pagini si marcheaza intrarea valida. Exceptia aruncata este numita **page fault**.

3.22 In ce situatie apare page fault fara a cauza segmentation fault?

- in cazul **demand paging**, se aloca un page, dar nu si un frame: la aparitia unui page fault se va aloca un frame, dar nu cauzeaza segmentation fault;
- in cazul in care avem un page valid, dar al carui corespondent se gaseste pe **disk**, un acces genereaza page fault: continutul este swapped intr-un frame si procesul isi continua executia;
- in cazul **copy-on-write,** doua page-uri sunt marcare read-only si refera acelasi frame; in momentul in care unul dintre cele doua procese efectueaza o operatie de scriere, se genereaza page fault, se duplica frame-ul si se continua executia;

3.23 Ce rol are spatiul de swap?

Este **spatiul localizat pe disc**, folosit temporar pentru stocarea informatiilor din RAM.

3.24 Cand are loc swap in si swap out?

In momentul in care spatiul fizic (RAM-ul) devine insuficient, se evacueaza (**swap out**) anumite pagini fizie. In momentul in care aceste pagini sunt necesare, sunt readuse in memoria RAM (**swap in**).

3.25 Care este rolul unui page fault? in ce conditii apare?

Page fault-ul este o exceptie generata de MMU, care apare atunci cand un proces incearca sa acceseze o pagina ce este marcata "nevalida" in tabela de pagini. Este de mai multe tipuri:

- **nu este in VAS** sau **nu are permisiuni suficiente**: segmentation fault;
- **minor page fault**: atunci cand pagina este in memorie, dar nu este mapata (nu este nevoie sa itereze prin disc), situatie intalnita la demand paging;
- **major page fault**: atunci cand pagina este in swap (e nevoie de o iteratie a discului si dureaza foarte mult).

3.26 Care sunt sectiunile/zonele din spatiul de adrese al unui proces?

.text (cod), rodata (constante), data (variabile statice initializate), bss (variabile statice neinitializate), heap (zona pentru alocari dinamice), biblioteci (cu subzone: text, rodata, data, bss), stiva (variabile locale).

3.27 Ce sectiuni ale unui executabil se pot inspecta doar in timpul rularii?

Stiva, heap-ul si bibliotecile dinamice

3.28 Care sunt zonele writable din spatiul de adrese al unui proces?

.data, .bss, stiva si heap-ul

3.29 De ce sunt avantajoase bibliotecile dinamice pentru spatiul de adrese al unui proces?

Pentru ca procesul va partaja zone de cod din cadrul bibliotecii dinamice cu alte procese. Ele sunt incarcate o singura data in RAM si apoi mapate in spatiul virtual al fiecarui proces care le foloseste.

3.30 Doua procese sunt pornite din acelasi executabil, ce zone din spatiul de adrese vor partaja?

Vor partaja bibliotecile dinamice, kernel space, zona de cod si rodata.

3.31 Se aloca un buffer a[100]. De ce a[105] NU va rezulta, in general, in Segmentation fault?

Pentru ca, in general, a[105] se afla in aceeasi pagina cu a[100].

3.32 In ce situatie a[300] rezulta in Segmentation fault?

Cand adresa $a + 300 * sizeof(a)$ se afla intr-o pagina nealocata sau fara permisiunile necesare.

3.33 Cate pagini ocupa char a[100] pentru sistem standard cu pagini de 4096 de octeti?

char a[100] ocupa 100 de octeti, adica $100/4096 = 0.024$ pagini. Deci, se va ocupa o pagina.

3.34 Cate pagini fizice aloca un apel mmap() care aloca 1MB? O pagina ocupa 4KB.

Apelul mmap() nu aloca nicio pagina fizica, ci doar rezerva spatiu virtual.

3.35 Cate pagini fizice aloca un apel calloc() care aloca 1MB? O pagina ocupa 4KB.

Vom avea nevoie de $1MB/4KB = 1046/4 = 256$ de pagini fizice.

3.36 Ce inseamna maparea unui fisier in memorie? De ce este avantajos sa mapam fisiere fata de folosirea read/write?

Este un mecanism prin care procesele acceseaza fisiere prin incorporarea lor directa in VAS. Adica, scrierea intr-o pagina virtuala conduce la scrierea in blocul corespunzator de pe disc al procesului. Are loc uzual pentru fisiere executabile si biblioteci partajate.

Operatiile cu fisierele sunt acum operatii cu memoria, nu mai sunt apeluri de sistem read/write.

Avantaje: overhead temporal scazut, deoarece nu se mai fac apeluri de sistem.

Dezavantaje: fisierele trebuie sa aiba dimensiunea stiuta pentru mapare, nu se poate creste dimensiunea

3.37 Cate page fault-uri se pot obtine in cazul operatiei *a = b?

- Daca nu sunt permisiuni de scriere in a $->$ 1 page fault
- Daca pagina corespunzatoare lui a nu este mapata in memoria fizica, dereferentierea va genera un page fault $->$ inca 1 page fault
- Daca pagina corespunzatoare lui b nu este mapata in memoria fizica, accesul va genera inca 1 page fault

Deci, se pot genera intre 0 si 3 page fault-uri.

3.38 Care este numarul maxim de page fault-uri pe care il poate genera expresia a = b + c?

Eu zic ca sunt 3. Prove me wrong.

3.39 Ce informatii sunt retinute in stiva? Ce variabile C?

Variabilele locale functiei, adresa de return, stack-frame-ul, parametrii functiei.

3.40 Ce se intampla la faza de loading (incarcarea unui executabil in memorie si crearea unui proces)?

In timpul fazei de loading, SO citeste executabilul de pe disk si mapeaza parti din el (zonele statice: text, data, rodata, bss) in memoria fizica.

3.41 Ce inseamna deturnarea fluxului de executie a unui program (control flow hijack)? De ce este acest lucru relevant pentru un atacator?

Cotrol-flow hijack este un tip de atac care urmareste schimbarea flow-ului normal al aplicatiei, si executarea de cod fraudulos prin vulnerabilitati precum buffer overflow, index out of bounds, integer overflow sau string formatting.

Este relevant pentru ca scopul final al unui atac de tip control-flow hijack este preluarea controlului (pornirea unui nou shell, preluarea controlului unui server web, etc).

3.42 Ce inseamna memory leak / memory disclosure? De ce este acest lucru relevant pentru un atacator?

Inseamna ca cineva care nu ar trebui sa aiba acces la niste date/adrese, are acces la ele.

Este util pentru atacator pentru a afla date personale sau alte portiuni de memorie de interes prin care poate ajunge la alte date/alte adrese sau pe care le poate suprascrie in vederea denaturarii fluxului.

3.43 De ce in general, preferam o impartire a spatiului virtual de adrese intre kernel space si user space? si nu un spatiu dedicat pentru kernel space?

Pentru ca zona kernelului este comuna intre procese. Cand se face context switch nu va fi nevoie sa se schimbe intrarile din page table asociate cu zona pentru kernel.

3.44 Ce secventa de cod C va duce la o exceptie de acces la memorie (de tip Segmentation fault)? De ce?

int *a = NULL; int b = *a;

Dereferentierea unui pointer NULL duce la seg fault, deoarece pagina corespunzatoare lui a nu este in VAS, deci, o scriere in aceasta duce la un acces invalid.

3.45 Cu ce difera o functie de o variabila intr-un executabil si/sau in cadrul spatiului de adrese al unui proces?

O variabila nu este executabia, ea se afla in zona .data, .rodata, .bss sau stiva. O functie este un simbol ce poate fi executat. Ea se afla in zona .text si are permisiuni de citire si executie.

3.46 Doua procese partajeaza o zona de memorie. Cum se manifesta acest lucru in tabelele de pagini ale celor doua procese?

Ambele procese vor avea mapata aceeasi memorie fizica la memoria virtuala. Intrari din ambele tabele de pagini ale celor 2 procese vor pointa catre aceleasi pagini fizice.

3.47 Putem avea mai multa memorie fizica decat dimensiunea maxima a spatiului virtual de adrese al unui proces? Dar invers?

Da, se poate. Spatiul de adrese virtual reprezinta limita maxima de memorie pe care un proces o poate accesa, dar aceasta nu inseamna ca acest proces poate accesa intreaga memorie fizica disponibila pe sistem.

Invers, dimensiunea maxima a spatiului de adrese virtual poate fi mai mare decat memoria fizica disponibila in sistem, insa in acest caz, sistemul poate utiliza paginare si swap pentru a suplini aceasta diferenta.

3.48 Ce zone de memorie se aloca static? Dar dinamic?

Stiva si heap-ul sunt alocate dinamic; text, data, rodata si bss sunt alocate static, inainte de inceperea executiei.

3.49 Ce se intampla cu o variabila modificata intr-un proces copil din perspectiva procesului parinte?

Procesele nu shareuiesc variabile, deci nu s-ar intampla nimic.

3.50 Ce reprezinta un loader? Ce rol are acesta?

Loader-ul este o componenta a SO care se ocupa cu incarcarea programelor si a bibliotecilor. Are rolul de a valida permisiuni, de mapare a executabilului de pe disk in memorie, de a copia argumentele in memoria virtuala, de a initializa registrii (spre exemplu, SP) si face jump la entry-point (*_start*).

3.51 La ce folosim apelul mprotect()? La ce mecanism de securitate putem face bypass folosind acest apel?

Apelul mprotect() este folosit la schimbarea permisiunilor unei zone de memorie. Nuj

3.52 in ce zona de memorie se afla o variabila globala, initializata cu valoarea 0?

In bss.

4 Compute

O sa adaug mai multe intrebari.

4.1 Ce este un proces?

Un proces este un program in executie. Orice proces are un spatiu de adrese si unul sau mai multe fire de executie. Putem avea mai multe procese ce executa acelasi program, dar oricare doua procese sunt complet independente.

4.2 Ce este un thread?

Thread-urile sunt o varianta simpla a proceselor pentru executarea de actiuni; se mai numesc si LWP (lightweight processes). VAS si descriptorii de I/O apartin procesului si sunt informatii partajate intre thread-uri.

4.3 Cu ce difera un thread de un proces?

- **relatie de incluziune**: procesele pot avea thread-uri, insa thread-urile nu pot avea procese;
- **memoria partajata**: procesele nu partajeaza resurse, pe cand thread-urile da (intreg VAS-ul procesului care le naste, mai putin TLS, stiva si registrii);
- **erori**: daca un proces isi incheie executia necontrolat din cauza unei erori, programul poate continua sa ruleze normal, pe cand daca un thread crapa, tot programul crapa;
- **overhead**: thread-urile sunt mai lightweight si pot fi planificate mai usor, de asemenea, ele sunt mai rapide prin timpul de creare si cel de context switch.

4.4 Cum este afectat spatiul virtual de adrese al unui proces in momentul crearii unui thread?

VAS-ul procesului primeste o noua zona de memorie, pentru stiva noului thread (vreo 8MB).

4.5 Ce zone de memorie au comune thread-urile unui proces si ce zone au specifice?

Au la comun tot spatiul de adrese si au specific: stiva, registrii si o zona dedicata numita TLS (thread local storage).

4.6 Ce contine PCB (Process Control Block)?

PID, VAS, timp de lucru pe procesor, fisiere deschise, starea procesului, registrii, SP-ul, FDT

4.7 Care sunt starile in care se poate gasi un thread?

NEW, READY, RUNNING, WAITING (sau BLOCKED), TERMINATED

4.8 Ce efect are apelul fork()?

fork() duplica spatiul de adrese al unui proces prin crearea unui proces copil. Apelul se intoarce de doua ori (o data in parinte cu PID-ul procesului copil si o data in copil cu valoarea 0).

4.9 Ce resurse partajeaza/nu partajeaza procesul parinte si procesul copil in cazul apelului fork()?

Initial, se partajeaza **tot** spatiul fizic: atat procesul parinte cat si procesul copil au un spatiu virtual propriu, implicit o tabela de pagini proprie. Dar aceste spatii virtuale refera acelasi spatiu fizic, adica tabelele de pagini au acelasi continut.

Procesul copil mai primeste, de asemenea, si o copie a FDT parintelui, cu descriptori de fisiere ce pointeaza catre aceleasi open file structures ca file descriptorii parintelui.

4.10 Cum modifica fork() si exec() spatiul virtual de adrese?

Atat fork() cat si exec() creeaza un spatiu nou de adrese, diferenta dintre cele doua fiind aceea ca: fork() creeazza un nou spatiu de adrese procesului copil, pe cand exec() goleste spatiul virtual de adrese al procesului curent (mai putin partea de kernel space) si il pregateste pentru a rula noul program.

4.11 Ce efect are apelul exec()?

Lanseaza in executie procesul creat cu fork(). Fork() creeaza copilul ca o copie a parintelui. Exec() ne va laa sa suprascriem acea copie cu orice dorim noi sa lansam spre executie. Exec() **invoca loaderul si incarca o noua imagine de executabil.**

4.12 Cate threaduri se pot gasi in starea RUNNING, READY si WAITING?

- RUNNING: numarul maxim de threaduri in acea stare este dat de numarul de procesoare
- READY, WAITING: se pot gasi oricate thread-uri, in limita resurselor sistemului.

4.13 Ce este o schimbare de context? Ce se intampla la o schimbare de context?

Cand planificatorul decide ca un proces paraseste procesorul, are loc o schimbare de context. Schimbarea de context inseamna salvarea informatiilor procesului anterior intr-o zona din SO si restaurarea informatiilor noului proces. Ea produce overhead si poate fi:

- voluntara: un proces decide sau cauzeaza schimbarea de context
  - un proces cedeaza de buna voie: yielding;
  - un proces executa o operatie blocanta (citire de la un dispozitiv care nu are inca date);
  - un proces isi incheie executia;
- nevoluntara: planificatorul SO-ului decide fortarea unui proces de pe procesor
  - un proces a stat prea mult timp pe procesor;
  - apare in sistem un procesor cu o prioritate mai mare.

4.14 Ce cauzeaza schimbari de context?

1. yield() - thread-ul care ruleaza il lasa voluntar pe urmatorul din coada sa ii ia locul;
2. expirarea cuantei de timp;
3. operatii blocante, thread-ul asteapta dupa I/O;
4. exit: thread-ul si-a incheiat treaba inainte de expirarea cuantei;
5. apare un thread cu o prioritate mai mare.

4.15 Ce este o schimbare de context voluntara si o schimbare de context nevoluntara?

- voluntara: thread-ul face o actiune despre care stie sigur ca va duce le inlocuirea sa de pe procesor;
- nevoluntara: nu este data de o actiune diercta, ci de ceea ce dicteaza planificatorul, pentru a mentine echitatea timpului petrecut ruland.

4.16 Ce sunt threadurile cu implementare user-level si thread-urile cu implementare kernel-level?

Kernel-level threads inseamna suport la nivelul sistemului de operare; thread-urile sunt folosite de planificatorul sistemului de operare, au cuanta de executie si sunt parte din coada READY. Kernel-level threads pot fi procesoarele sistemului: mai multe thread-uri ale aceluiasi proces pot rula simultan pe un sistem multi-procesor. Aici, schimbarea de context necesita interventia nucleului: e nevoie de suport in kernel. **Avantaje**: suport complet multi-procesor, bune pentru actiuni CPU-intensive.

User-level threads sunt o implementare completa in user-space (inclusiv planificatorul). O operatie blocanta a unui user-level thread blocheaza tot procesorul. Solutia este sa se foloseasca operatii I/O asincrone. **Avantaje**: nu necesita suport in kernel, timp de activare scurt, performanta ridicata pentru actiuni I/O intensive.

4.17 in ce situatie este utila zona TLS (thread local storage)?

Cand dorim sa avem variabile globale, a.i. sa fie accesibile de toate functiile, dar nu dorim ca acele variabile sa fie accesibile intre thread-uri.

4.18 De ce este necesara sincronizarea proceselor/thread-urilor?

Pentru a ne asigura ca nu exista situatii in care programul are caracter nedeterminist.

4.19 Ce inseamna race condition?

O problema ce poate aparea cand scriu 2 thread-uri si este posibil ca operatiile de scriere sa nu fie atomice, sa se intercaleze si sa rezulte in date inconsecvente: de exemplu, daca un thread face a++ si altul face a++, e posibil ca rezultatul sa fie ainitial + 1, in loc de ainitial + 2.

Deci, race condition se refera la **rezultatul nedeterminist** pe care o are actiunea facuta de doua sau mai multe thread-uri simultan, dorind sa modifice o resursa comuna.

4.20 Ce inseamna deadlock?

Deadlock-ul este rezultatul actiunii prin care doua thread-uri asteapta ceva din partea unui altuia. Cu alte cuvinte, **thread-urile depind unul de altul si nu isi pot continua executia**, deoarece niciunul dintre ele nu poate iesi din loop.

4.21 Care sunt dezavantajele sincronizarii?

- greu de implementat overall
- ineficienta: deadlock-uri, regiuni critice mari;
- overhead mare: la instructiuni atomice (lock pe magistrala, un singur procesor acceseaza memoria), la spinlock (busy waiting ocupand loc pe procesor), la mutex/semafor (apel de sistem, context switch).

4.22 Ce inseamna TOCTOU (time of check to time of use)?

Se refera la un race condition care apare imediat dupa un compare, inainte de a se face instructiunea urmatoare (de folosire), cand un alt thread apare si modifica valoarea considerata "safe".

4.23 Cand se blocheaza un producator in problema producator - consumator? Dar un consumator?

Un **producator** se blocheaza cand **bufferul comun este plin** si nu are unde sa mai scrie date. Un **consumator** se blocheaza cand **bufferul comun este gol** si nu are ce date sa mai citeasca.

Nota: **Problema producator-consumator** se refera la niste componente ce comunica intre ele (de exemplu: placa de retea-procesor, server-client, dispozitiv I/O-memorie). O componenta produce informatia, cealalta o consuma. Se intampla deseori ca una sa fie mai rapida decat cealalta. Pentru comunicatie, se foloseste un **buffer cu doua capete,** unul pentru citire, altul pentru scriere. Daca e lent consumatorul (citirea se face greu), producatorul are spatiu de depunere. Daca e lent producatorul (scrierea se face greu), consumatorul poate consuma ce s-a scris pana la acel moment in buffer.

4.24 De ce este necesara prezenta unei instructiuni de tipul atomic_compare_and_swap in fiecare ISA?

O instructiune de tipul Atomic CAS sta la baza tuturor mecanismelor de sincronizare. Daca CAS nu ar fi garantat atomica, nici mutex-urile, nici semafoarele sau spinlock-urile nu ar putea exista.

4.25 Cu ce difera un spinlock de un mutex? Cand folosim spinlock-uri? Cand folosim mutex-uri?

Spinlock-ul este primitiva de baza pentru asigurarea atomicitatii. Face **busy-waiting** in asteptarea eliberarii si se bazeaza pe CAS. Spinlock-ul este util pentru **regiuni critice de mici dimensiuni.**

Mutex-ul este o primitiva mai complexa, cu o **coada de thread-uri in asteptare**. Daca mutex-ul este luat, thread-ul intra in sleep. Pentru protejarea structurilor interne, un mutex are un spinlock. Mutex-ul este util pentru **regiuni critice mari** sau unele in care thread-ul se blocheaza.

4.26 Ce efect are folosirea operatorului din shell in crearea unui proces?

Daca rulam o comanda cu la sfarsit (pentru rulare in background), shell-ul nu asteapta incheierea comenzii (nu mai face waiting blocant).

4.27 Ce este un proces zombie? Cum apare un proces zombie? Care este problema proceselor zombie?

Un proces zombie este un proces care **si-a incheiat executia dar nu a fost asteptat de parintele sau**. Problema poate fi cauzata de faptul ca ele **ocupa resurse utile** (PID, intrare in tabela de procese) **degeaba**, deoarece acestea nu mai au nimic de executat, dar nu sunt eliberate.

4.28 Ce este un proces orfan? De ce un proces este orfan foarte putin timp?

Un proces orfan este un proces al carui **proces parinte s-a incheiat**. El este orfan putin timp, deoarece ajunge sa fie infiat de procesul init.

4.29 Poate fi un proces zombie orfan? Ce se intampla cu un proces zombie orfan?

Daca un proces parinte moare si nu a asteptat un proces copil zombie, acesta este zombie orfan. Este infiat de procesul init si apoi este "curatat" din sistem.

4.30 Ce se intampla daca un thread realizeaza un acces invalid la o zona de memorie?

Thread-ul, impreuna cu intreg procesul de care tine, primeste segmentation fault, se apeleaza o rutina de tratare a exceptiei si programul se incheie cu o eroare.

4.31 Poate un thread sa acceseze stiva altui thread? Cum?

Da. Se poate retine intr-o **variabila globala o adresa de stiva a altui thread** (pentru thread-urile aceluiasi proces), sau **se iau adrese random**: daca adresa este valida, totul merge bine, daca nu, segfault.

4.32 De ce schimbarea de context intre doua thread-uri ale aceluiasi proces este mai rapida decat schimbarea de context intre doua thread-uri din procese diferite?

Avand in vedere ca thread-urile aceluiasi proces share-uiesc memoria, nu este necesar flush-ul TLB-ului, asa cum ar fi la schimbarea de context intre doua thread-uri din procese diferite.

4.33 Ce limiteaza numarul maxim de threaduri care pot fi create in cadrul unui proces?

Dimensiunea memoriei virtuale, deoarece pentru fiecare nou thread trebuie creata o noua stiva in cadrul VAS-ului procesului.

4.34 Ce limiteaza numarul maxim de procese care pot fi create in cadrul unui sistem de calcul?

Numarul maxim de intrari in tabela de procese.

4.35 Ce se intampla cand toate procesele sistemului sunt blocate?

Apare un proces special numit IDLE, care face busy waiting pana un alt proces este pregatit sa intre pe procesor.

4.36 Ce inseamna waiting time (timp de asteptare) in planificarea proceselor?

Este timpul pe care il petrec procesele in starea READY. Ideal, acesta este cat mai mic.

4.37 Putem avea un sistem multi-core cu un singur proces aflat in starea RUNNING si mai multe procese in READY?

DA, daca prin proces se intelege un proces cu mai multe thread-uri: in acest caz, toate thread-urile procesului pot rula in acelasi timp, ocupand toate procesoarele si nepermitand altor taskuri sa ruleze.

4.38 Ce inseamna starea READY/RUNNING/TERMINATED/WAITING (BLOCKED)?

- **READY** → thread-ul este gata pentru a intra pe procesor, asteapta sa i se faca loc, sa fie planificat;
- **RUNNING** → thread-ul ruleaza, are o cuanta stabilita si poate executa oricate actiuni pe parcursul ei;
- **TERMINATED** → thread-ul si-a incheiat executia si face exit();
- **WAITING** → thread-ul a facut un apel blocant si asteapta dupa I/O;

4.39 Ce este un proces I/O intensive?

Un proces I/O intensive este un proces care lucreaza mult cu I/O-ul si, implicit, are multe apeluri blocante. De cele mai multe ori, cand un astfel de proces iese din RUNNING, trece in BLOCKED.

4.40 Ce este un proces CPU intensive?

Un proces care face calcule aritmetice, interactionand foarte des cu procesorul. De cele mai multe ori, cand un astfel de proces iese din RUNNING, trece in READY.

4.41 Cum trateaza planificatorul procesele I/O intensive si procesele CPU intensive?

In general, procesele I/O intensive primesc o prioritate mai mare decat cele CPU intensive.

Cele I/O intensive primesc o cuanta mai mare de timp, deoarece ele ajung voluntar in BLOCKED, nu au tendinta de a acapara CPU-ul.

Cele CPU intensive primesc o cuanta mai mica pentru a mentine waiting time-ul mic.

4.42 Doua thread-uri ale unui proces executa aceeasi functie. Care sunt diferentele intre cele doua thread-uri?

Au stive diferite si instruction pointeri diferiti.

4.43 Cu ce difera procesul copil (creat prin fork) de procesul parinte?

Difera prin PID, prin VAS (dupa write, caci initial sunt identice).

4.44 Cu ce difera un proces zombie de un proces orfan?

Procesul zombie poate avea procesul parinte in viata.

4.45 Ce parametri ai planificatorului trebuie sa modificam pentru a avea un sistem cu productivitate mai mare?

Se pot da cuante de timp mai mari proceselor pentru a minimiza context switch-urile.

4.46 Ce parametri ai planificatorului trebuie sa modificam pentru a avea un sistem cat mai interactiv (responsive)?

Procesele trebuie sa primeasca o cuanta de timp cat mai mica, pentru a avea mentine un waiting time mic.

4.47 Am avea nevoie de folosirea unui apel de sistem pentru crearea unui thread in cazul unei implementari de tip user-level threads? Dar in cazul deschiderii unui fisier in acelasi scenariu?

Nu este necesar un apel de sistem pentru crearea unui thread in user space, deoarece putem utiliza conceptul de green threads (fire de executie care ruleaza la nivel de bibiloteca).

Este necesar un apel de sistem pentru deschiderea unui fisier, deoarece, in acest caz, interactionam cu disk-ul si avem nevoie de permisiunea kernelului pentru a face asta.

4.48 Ce se intampla daca folosim apeluri de sistem blocante in interiorul unui spinlock?

Ar duce la deadlock.

4.49 De ce este necesara folosirea prefixului LOCK pentru realizarea operatiilor atomice?

Prefixul LOCK anunta SO sa faca un lock pe magistrala. Ramane un singur core ce va avea acces la memorie, deci operatia atomica va putea sa aiba loc.

4.50 Care este avantajul folosirii mutexurilor in defavoarea spinlockurilor si invers?

Mutex-urile sunt folosite pentru regiuni critice mari, cu apeluri blocante, pe cand spinlock-urile sunt ieftine cand vine vorba de lock() si unlock(), pentru ca nu fac context switch si nu invoca planificatorul.

4.51 Care este echivalentul apelului wait() pentru threaduri?

Apelul join() este echivalent apelului wait(). El permite unui thread sa astepte terminarea altui thread.

4.52 De ce este nevoie de apelul pthread_wait() / pthread_join()?

Apelurile pthread_wait() / pthread_join() realizeaza eliberarea resurselor folosite de proces/thread dupa terminarea acestora.

4.53 Ce tranzitie intre starile unui thread nu este posibila?

READY $\rightarrow$ BLOCKED: thread-ul trebuie mai intai sa fie capabil sa faca o actiune pentru a putea face un apel blocant, iar asta se intampla numai daca este running.

5 I/O

5.1 Ce contine un FCB (File Control Block)?

Metadatele unui fisier (tipul fisierului, id, permisiuni, dimensiune, owner, group, timpi de acces, numarul de link-uri) se gasesc intr-o structura numita FCB (numita si inode pe Unix).

5.2 Ce reprezinta un descriptor de fisier?

Este un **numar intreg** ce refera o intrare in tabela de descriptori de fisier. Este folosit in operatii cu lucru cu fisiere, pentru a identifica un fisier deschis.

5.3 Ce reprezinta tabela de descriptori de fisiere?

Tabela de descriptori de fisiere este retinuta in kernel space si exista cate o astfel de **tabela pentru fiecare proces**. Ca structura de date, ea este un **vector de pointeri**; acesti pointeri refera fisierele pe care un proces le utilizeaza. Aceste tabele sunt populate prin apeluri precum open(), pipe(), dup(), create(), socket() sau accept().

5.4 Cate tabele de descriptori de fisiere se gasesc intr-un sistem de operare?

Cate una pentru fiecare proces.

5.5 Ce efect are apelul dup()?

- apelul **dup(fd)** face ca prima intrare libera gasita in tabela de descriptori de fisiere sa refere structura de fisier deschis referita de fd;
- apelul **dup2(olfd, newfd)** face ca intrarea cu indexul newfd sa refere structura de fisier deschis referita de oldfd.

5.6 Ce efect are apelul close()?

Elibereaza file descriptor-ul asociat. Scade numarul de link-uri catre un open file structure. Daca numarul ajunge sa fie 0, atunci open file structure-ul este eliberat la randul sau.

5.7 Ce apeluri modifica pointer-ul/cursorul de fisier (file pointer)?

open(), write(), read(), lseek(), ftruncate() (cursorul este plasat la 0);

5.8 Ce apeluri modifica dimensiunea fisierului?

open(), write(), ftruncate()

5.9 Ce efect are apelul/comanda truncate?

Seteaza dimensiunea fisierului la size-ul care este dat ca argument.

5.10 Care sunt tipurile de fisiere pe un sistem de fisiere uzual Unix?

regular, directory, symlink, named pipes (FIFO), block device, char device, socketi UNIX

5.11 Ce este un sistem de fisiere virtual?

Este o interfata abstracta pe care SO o expune catre sistemele de fisiere traditionale (ext3, VFAT, FreeBSD etc) a.i. diferentele interne de implementare fiecarui sistem de fisiere sa nu fie o problema, kernelul dobandind o modalitate de comunicare facila cu multiple sisteme de fisiere simultan.

5.12 Ce este un dispozitiv virtual?

Un dispozitiv virtual mimeaza existenta unui dispozitiv hardware fizic. Kernelul il identifica drept un periferic, insa acesta este doar un fisier/director. Un astfel de dispozitiv virtual se poate crea cu apelul "mknod()".

5.13 Ce tipuri de dispozitive cunoasteti? Clasificati-le din orice punct de vedere cunoasteti

- char devices (dispozitive seriale): se ocupa cu un singur caracter la un moment dat (ex: tastatura, mouse, terminal);
- block devices (random access devices): se ocupa cu blocuri de date cu dimensiunea multiplu de 512 (ex: disk, CD-ROM, flash drives);

5.14 Cu ce difera un dispozitiv de tip bloc de un dispozitiv de tip caracter? Dati cate un exemplu de fiecare.

- char devices: nu se face buffering; apelurile read si write sunt blocante; accesul in cadrul acestui tip de device este mai simplu: necesita mai putin efort, deoarece are de gestionat o singura pozitie (cea curenta);
- block devices: accesate printr-un cache; read si write sunt asincrone; driverul asociat kernelului necesita mai mult efort, pentru ca trebuie sa treaca printr-un buffer cache pentru a scrie si citi;

5.15 De ce nu are sens operatia de seek pe un dispozitiv de tip caracter?

Deoarece un char device are o singura pozitie, cea curenta, datele venind sub forma "one char at a time", asadar nu putem misca un pointer prin fisier.

5.16 Ce adresa IP locala si ce port local are un socket intors de apelul accept()?

Apelul accept() intoarce un fd pentru un socket cu adresa IP a retelei locale si portul aplicatiei care doreste sa se conecteze.

5.17 Ce valoare poate intoarce un apel read() sau un apel write()?

- read(): numarul de bytes cititi, 0 pentru EOF sau -1 la eroare;
- write(): numarul de bytes scrisi sau -1 la eroare.

5.18 Ce operatii se pot face pe fisiere?

create(), open(), read(), write(), lseek(), truncate(), close(), delete

5.19 Ce operatii asupra fisierelor modifica/nu modifica valoarea cursorului unui fisier?

- modifica cursorul: open, read, write, lseek
- NU modifica cursorul: stat, truncate

5.20 Ce operatii asupra fisierelor modifica/nu modifica dimensiunea fisierului?

- modifica dimensiunea: open, write, truncate
- NU modifica dimensiunea: read, lseek

5.21 Unde este retinuta valoarea cursorului de fisiere (file pointer) si unde este retinuta dimensiunea fisierului?

Dimensiunea fisierului e retinuta in structura inode-ului / FCB (in metadate), iar file pointer-ul e retinut in open file structure.

5.22 De ce avem doua buffere asociate fiecarui socket, ce rol are fiecare?

Socket-ul, spre deosebire de pipe, permite comunicarea **bidirectionala**. Pentru a asigura faptul ca sender-ul si receiver-ul nu trimit informatie dintr-o parte in alta in acelasi timp, pe acelasi canal, exista doua buffere: **unul de send si altul de receive**.

5.23 Ce este o operatie asincrona?

O operatie asincrona e o operatie care nu se executa neaparat in momentul in care apare. In cazul in care nu se poate efectua imediat, ea se blocheaza si asteapta sa fie rezolvata.

5.24 Ce este o operatie neblocanta?

O operatie neblocanta permite executarea altor operatii, chiar daca nu a ajuns la finalul executiei.

5.25 Ce intoarce o operatie asincrona?

Operatiile asincrone nu intorc date, ci un cod de eroare, 0 sau 1. Datele pot fi accesate printr-un eventfd().

5.26 Ce intoarce o operatie blocanta?

Operatia blocanta intoarce rezultatul actiunii; tocmai de aceea este blocanta, pentru ca asteapta mereu pana la incheierea task-ului.

5.27 Cu ce difera un socket de retea de un socket UNIX?

Socket-ul de retea este folosit pentru comunicarea proceselor de pe sisteme diferite. Socket-ul UNIX este o forma de comunicare intre procese aflate pe acelasi sistem.

5.28 Care este diferenta intre un pipe anonim si un pipe cu nume (named pipe)?

Pipe-ul cu nume permite comunicarea intre doua procese **neinrudite**, pe cand prin **pipe-ul anonim** se poate transmite informatie doar **intre procese inrudite**. Mai mult, pipe-ul cu nume apare ca o intrare in sistemul de fisiere si reprezinta chiar un tip de fisier (FIFO).

5.29 Ce este buffer cache-ul? Care este rolul sau?

La lucrul cu fisiere de pe disc, apelurile read/write nu acceseaza direct disk-ul, ci copiaza datele utilizate recent, dintr-un buffer din user space intr-unul din kernel space, numit buffer cache, cu scopul de a reduce overhead-ul temporal.

5.30 De ce operatia write pe fisiere este foarte rar blocanta?

Operatia de write() ar fi blocanta daca buffer-ul din kernel space ar fi plin. Acesta este extrem de rar plin, pentru ca daca se umple, este fie eliberat (prin scrierea pe disk a informatiilor pe care le retine), fie marit.

5.31 in ce situatie operatia read() pe fisier se blocheaza?

Operatia de read() este blocanta in cazul in care buffer-ul din kernel space este gol. Actiunea va fi blocata pana cand cel putin un octet ajunge in buffer.

5.32 Care este rolul unui device driver?

Device driver-ul este o bucata de software folosita pentru a controla diferite componente hardware. Are rolul de a permite sistemului de operare sa se foloseasca de periferice.

5.33 Ce rol are controller-ul hardware?

Controleaza componentele hardware (are rol de administrator). Este la randul sau o componenta hardware care permite transmiterea, flow-ul de date, intre mai multe entitati fizice.

5.34 Ce inseamna zero-copy? Ce mecanism/apel foloseste zero-copy?

Zero-copy se refera la transferul de date intre mai multe buffere din kernel space fara a se trece prin user space. Interactiunea cu aplicatia devine minima, iar costul temporal adus de domain switch este evitat.

Apeluri folosite pentru zero-copy sunt sendfile(), splice() (UNIX), TransmitFile (Windows).

5.35 Ce garantii ni se ofera in momentul in care apelul send() se intoarce in user space?

Datele au fost copiate intr-un send buffer.

5.36 Cu ce difera afisarea folosind printf() fata de folosirea write()?

printf() afiseaza implicit la stdout, pe cand lui write() trebuie sa ii precizezi fd-ul corespunzator.

5.37 De ce subsistemul de networking nu foloseste buffer cache-ul?

Pentru ca buffer cache-ul este un cache exclusiv pentru lucrul cu disk-ul. Astfel, buffer cache-ul nu ar avea sens pentru subsistemul de networking care functioneaza pe baza de pachet, ca unitate fundamentala de comunicare.

5.38 Ce rol are apelul / comanda sync?

sync() realizeaza sincronizarea dintre bufferele din kernel space cu ceea ce rezida pe disk.

5.39 Care este rolul apelului ioctl / DeviceIoControl?

ioctl() este un apel de sistem ce se ocupa cu **comunicarea dintre aplicatii si device drivere**. El poate fi folosit pentru mai multe tipuri de fisiere, precum char/block devices, sockets, in general tot ceea ce are un file descriptor.

5.40 De ce in general doar utilizatorul root are permisiuni de scriere (uneori doar root are permisiuni de citire) pe intrarile din /dev?

Pentru ca interactionarea cu device-urile hardware se face privilegiat. Astfel, user-ul nu poate aduce riscuri de securitate, iar comunicarea este lasata in mana kernel-ului, aceasta ramanand functionala si sigura.

5.41 De ce este apelul fwrite() mai rapid decat write atunci cand facem multe scrieri?

fwrite() fiind buffered, devine mai rapid la mai multe scrieri: acesta le va combina pe toate intr-un singur mare apel de sistem, in loc sa faca pentru fiecare actiune de scriere un syscall separat.

5.42 Ce se intampla daca facem open de mai multe ori consecutiv pe acelasi fisier?

Se va adauga un nou file descriptor in FDT si se va crea un alt open file structure pentru fd-ul creat.

5.43 Cum se modifica tabelul de file descriptori dupa open() si dup()? Este echivalent cu doua apeluri open()?

open() creeaza un nou file descriptor si un nou open file structure in FDT, pe cand dup() creeaza un nou file descriptor si point-eaza catre acelasi open file structure de la vechiul fd, pe care il primeste ca argument, incrementand numarul de referinte (link-uri) din structura.

dup() nu este neaparat echivalent cu doua apeluri open() pentru ca nu creeaza un open file structure; el doar copiaza referinta dintr-o parte a vectorului de fd-uri in alta.

5.44 Ce fisiere sunt deschise, in general, la crearea unui proces nou?

stdin, stdout, stderr cu file descriptorii 0, 1, 2.

5.45 De ce este utila prezenta unor dispozitive pur virtuale in ierarhia /dev (ex. /dev/vboxnetctl, /dev/urandom)?

Pentru a putea emula un tip de dispozitiv hardware fara a avea la indemana piesa respectiva; pentru a putea avea functionalitati precum cele date de urandom si random, pentru care nu exista un echivalent fizic.

5.46 De ce, in general, apelul write() pe un fisier nu blocheaza threadul curent?

Pentru ca realizeaza un simplu memcpy() intre bufferele din user space si cele din kernel space.

5.47 In ce situatie un apel read() pe un fisier blocheaza threadul curent?

Atunci cand buffer-ul din kernel space e gol.

5.48 De ce spunem ca operatiile read() / write() seamana cu operatia memcpy()?

Operatiile de read() si write() seamana cu memcpy() deoarece doar trimit informatia dintr-o parte in alta (buffere US - KS) prin copiere.

6 Application Interaction

6.1 Ce forme de comunicare inter-proces cunosti?

- comunicare: pipe-uri, fisiere, sockets, shared memory
- sincronizare: prin semafoare cu nume
- semnale

6.2 Ce este un semnal? Cand se trimite un semnal catre un proces?

Semnalul este un mesaj trimis asincron proceselor pentru a declansa un anumit comportament. La primirea semnalului este rulat un handler specific care trateaza eroarea, intrerupe, suspenda sau omoara procesul.

6.3 Cine trimite un semnal unui proces?

Sistemul de operare, alt proces sau chiar el insusi.

6.4 Cum este implementat operatorul | din shell?

— este implementat similar cu pipe() din C: este folosit un buffer comun in care primul proces trimite informatie, iar cel de-al doilea o primeste. Cele doua capete ale buffer-ului corespund celor doua procese care comunica.

6.5 Care sunt avantajele si dezavantajele folosirii memoriei partajate pentru comunicarea inter-proces?

- **Avantaje:** mai rapida decat orice alt tip de comunicare, ieftina ca memorie, este evitata folosirea si copierea intre multiple buffere;
- **Dezvantaje:** sincronizarea trebuie facuta ”de mana” de catre programator.

6.6 Care sunt avantajele si dezavantajele pipe-urilor pentru comunicarea inter-proces?

- **Avantaje:** comunicare facila si relativ rapida;
- **Dezvantaje:** comunicarea nu e bidirectionala (exista doar sender si receiver) si dimensiunea limitata a buffer-ului.

6.7 Care este limitarea folosirii pipe-urilor anonime?

Pot intretine comunicarea doar intre procese inrudite. De asemenea, sunt dependente de procesele asociate. Un pipe anonim este sters dupa terminarea procesului, spre deosebire de FIFO-uri care au un back-up in sistemul de fisiere virtual.

6.8 Care este diferenta principala intre FIFO si socket UNIX?

FIFO-urile au un singur canal de comunicare (unidirectional), pe cand socket-urile UNIX prezinta doua buffere (comunicare bidirectionala), unul pentru send si unul pentru receive.

6.9 Ce primitive de interactiune pot fi folosite intre procese de pe sisteme diferite?

Doar socket-uri de retea.

6.10 Ce primitive de comunicare pot fi folosite doar intre threaduri?

Doar shared memory.

6.11 Ce primitive de comunicare pot fi folosite doar intre procese inrudite?

Doar pipe-uri anonime.

6.12 Cum pot doua procese sa foloseasca semafoare anonime?

Doua procese inrudite pot folosi acelasi semafor anonim daca la crearea acestuia este pus 1 pentru argumentul de pshared.

6.13 Cum pot doua procese neinrudite sa foloseasca semafoare anonime?

Doua procese neinrudite pot folosi acelasi semafor anonim daca acesta este plasat intr-o zona de memorie pe care procesele o share-uiesc.

6.14 Ce rol are apelul mmap() in cazul partajarii memoriei intre doua procese?

mmap(), cu flag-urile specifice, creeaza zone de memorie anonime ce pot fi folosite de procese inrudite. Mai mult, mapeaza fisiere ce folosesc atat mecanismele de demand paging cat si de lazy loading pentru o folosire cat mai eficienta a memoriei.

6.15 Ce mecanisme putem folosi pentru a asigura accesul exclusiv la o zona de memorie partajata intre doua procese?

Sincronizarea proceselor este mecanismul prin care asiguram acces exclusiv: vom folosi mutex-uri (retinute in zone partajate de memorie) sau semafoare (mult mai usor cu semafoare, pentru ca putem folosi semafoare cu nume).

6.16 De ce este nevoie de sincronizare la folosirea memoriei partajate dar nu la folosirea unui socket pentru comunicarea intre doua procese?

Pentru ca sincronizarea zonelor de memorie partajata este imposibil de asigurat de kernel. Acesta, insa, garanteaza sincronizarea socket-urilor si a celorlalte tipuri de comunicare prin stream-uri.

6.17 Ce inseamna aplicatie omogena? Dati un exemplu.

O aplicatie omogena foloseste thread-uri/procese pentru a realiza acelasi tip de actiune concomitent. De obicei, astfel de aplicatii dau dovada de interactiune minimala intre thread-uri/procese, acestea doar dau join/wait la finalul executiei job-ului.

ex: multi-threaded web servers, libx264 din ffmpeg (thread-uri), paginile de browser din firefox (thread-uri), paginile de browser din chrome (procese).

6.18 Ce inseamna aplicatie eterogena? Dati un exemplu.

O aplicatie eterogena foloseste thread-uri/procese pentru a realiza tipuri diferite de actiuni concomitent. De obicei, astfel de aplicatii dau dovada de interactiune bogata intre thread-uri/procese, sunt folosite mecanisme de IPC.

ex: postfix, gitlab.

6.19 Care este avantajul si dezavantajul distribuirii unei aplicatii pe mai multe sisteme?

- **Avantaje:** se evita incarcarea sistemului propriu, putere computationala mai mare (sisteme distribuite);
- **Dezvantaje:** comunicare mai greoaie intre componente (se pot folosi doar sockets de retea, mai scumpi ca timp).

6.20 Care este avantajul implementarii unei aplicatii in format multi-process fata de format multi-threaded?

- **Securitate:** procesele nu pot accesa facil memoria unui altuia;
- **Robustete:** daca un proces crapa, nu crapa tot programul (asa cum s-ar intampla la thread-uri);
- **Scalabilitate:** poti avea mai multe procese ale aceleiasi aplicatii ruland concomitent pe mai multe sisteme.

6.21 Cum pot doua procese partaja memorie anonima? Adica folosind flagul MAP_ANONYMOUS la mmap().

Folosirea MAP_ANONYMOUS impreuna cu MAP_SHARED are urmatoarele efecte:

- maparea si zeroizarea unei zone distincte de memorie;
- copiii mostenesc zonele mapate ale parintilor;
- nu exista copy-on-write cand unul dintre procese incearca sa scrie;
- zonele mapate anonim sunt folosite ca mecaniscm de IPC, ceea ce numim shared memory;

6.22 Ce mecanism de interactiune intre aplicatii au un buffer si ce mecanisme de interactiune nu au un buffer?

- **cu buffer:** pipes, sockets;
- **fara buffer:** fisiere, shared memory, mutex-uri, semafoare, semnale.